

# 119.1. La fonction sauver() — garder un nombre, même éteint

**#include <sauver>** : sauver(numero, valeur) écrit dans la mémoire de la cartouche, qui garde tout quand on éteint.

## 1. Ce qu'il faut comprendre

**Ce qu'elle fait** : la cartouche porte une **pile** et une petite mémoire. `sauver(numero, valeur)` y écrit une valeur dans une de ses **256 cases** : elle y sera encore après avoir éteint la console. C'est ainsi qu'on garde un record.

**La ligne à écrire** : `#include <sauver>`.

**Ses arguments** : le numéro de la case (0 à 255), la valeur.

**Ce qu'elle coûte** : une vingtaine d'octets ; la cartouche s'annonce avec une mémoire à pile.

**Essaie** : sauve le score à chaque fois qu'il change.

**On retrouve sauver dans 3 leçons**

[120](#), [135](#), [138](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **322 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                |      |                                                                       |
|----------------|------|-----------------------------------------------------------------------|
| démarrage      | 49 o | ← la console, à l'allumage — la mise en route de la console           |
| main()         | 42 o | ← démarrage — ta fonction                                             |
| VBlank         | 8 o  | ← la console, à chaque image — compte les images, 60 fois par seconde |
| AttendreImage  | 28 o | ← image() — attend l'image suivante                                   |
| AttendreVBlank | 16 o | ← démarrage — attend que l'écran soit libre                           |
| AttendreAcces  | 12 o | ← EcrireTexte — attend de pouvoir écrire une case                     |
| EcrireTexte    | 10 o | ← texte() — écrit un texte, lettre par lettre                         |
| CopierTuiles   | 58 o | ← démarrage — copie les lettres et les dessins en mémoire vidéo       |
| EffacerCarte   | 14 o | ← démarrage — vide les deux cartes de l'écran                         |
| Tuiles         | 80 o | ← CopierTuiles — 5 lettres sur 43                                     |
| "SAUVE"        | 5 o  | ← texte() — un texte gravé                                            |

Lettres gravées : AESUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

|          |     |                                                   |
|----------|-----|---------------------------------------------------|
| sauver() | × 1 | 24 o sur place, dans main()                       |
| texte()  | × 1 | 11 o sur place, dans main() + EcrireTexte (10 o)  |
| image()  | × 1 | 3 o sur place, dans main() + AttendreImage (28 o) |

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

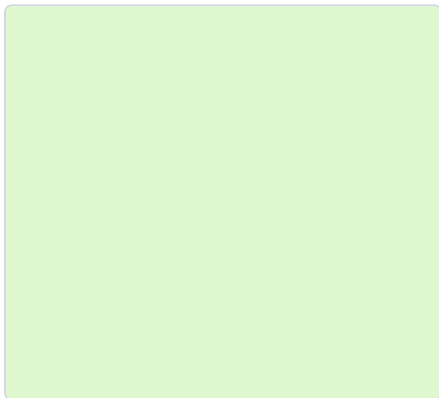
```
// ---- #include <sauver> : garder un nombre, même éteint ----  
  
#include <sauver>    // sauver() : écrire dans la mémoire de la cartouche  
#include <texte>
```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    sauver(0, 42);        // la case 0 de la cartouche reçoit 42  
    texte(5, 8, "SAUVE");  
  
    while (true) {  
        image();  
    }  
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

SAUVE écrit ; 42 est rangé dans la cartouche.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche annonce une mémoire à pile
- ✓ la case 0 de la cartouche vaut 42 (42)

Ensuite : **119.2. La fonction sauvegarde() — relire un nombre gardé** — chapitre 20 : Les états du jeu : titre, partie, fin. ·  
gameboy3 — écrire une cartouche Game Boy en C++